
SPARe: Stacked Parallelism with Adaptive Reordering for Fault-Tolerant LLM Pretraining Systems with 100k+ GPUs

Jin Lee^{1 2} Zhonghao Chen³ Xuhan He³ Robert Underwood² Bogdan Nicolae²
Franck Cappello² Xiaoyi Lu³ Sheng Di² Zheng Zhang¹

Abstract

In large-scale LLM pretraining systems with 100k+ GPUs, failures become the norm rather than the exception, and restart costs can dominate wall-clock training time. However, existing fault-tolerance mechanisms are largely unprepared for this restart-dominant regime. To address this challenge, we propose SPARe—Stacked Parallelism with Adaptive Reordering—a fault-tolerance framework that masks node failures during gradient synchronization by stacking redundant data shards across parallelism groups and adaptively reordering execution. SPARe achieves availability comparable to traditional replication while maintaining near-constant computation overhead of only $2 \sim 3\times$, even under high redundancy where traditional replication would require linearly inflating overhead. We derive closed-form expressions for endurable failure count and computation overhead, validate them via discrete-event simulation, and jointly optimize redundancy and checkpointing to minimize time-to-train. At extreme scale with up to 600k GPUs, SPARe reduces time-to-train by $40 \sim 50\%$ compared to traditional replication.

1. Introduction

The industrial high-performance computing (HPC) system has scaled up to over 100k+ GPUs to meet the computing demand of large-scale pre-training of AI foundation models. As the number of GPUs ($\#GPU$) increases, the mean time between failures (MTBF) of system decreases as

¹Department of ECE, University of California, Santa Barbara, United States ²Argonne National Laboratory, Illinois, United States ³Department of ECE, University of Florida, United States. Correspondence to: Jin Lee <hojin@ucsb.edu>, Sheng Di <sdi1@anl.gov>, Zheng Zhang <zhengzhang@ece.ucsb.edu>.

$\mathcal{O}(\frac{1}{\#GPU})$ (Kokolis et al., 2025; Ostrouchov et al., 2020). Meanwhile, upon system failures, the restarting latency inflates quadratically or worse by default due to global collectives initialization and synchronization (Jiang et al., 2024; Si et al., 2025). As a result, cumulative waste of system downtime from frequent failures grows rapidly, leading to lower system availability¹ and longer time-to-train². The LLaMa-3 paper (Dubey et al., 2024) (Section 3.3.4) reports that the average failure rate is around 4 interruptions per 1000 servers (8 GPUs per server) per day. This corresponds to one failure every three hours on average for a 16k H100-GPU system. Extrapolating using the empirically validated MTBF scaling model in Kokolis et al. (2025), failures would occur every 30 minutes at 96k GPUs and every five minutes at 600k GPUs. This projection is already harsh, yet still optimistic, as modern pretraining systems are increasingly heterogeneous, which further exacerbates failure rates (Jones, 2020; Jayaram Subramanya et al., 2023).

As failure-induced global restarts now happen once per hour or more, *collective communication (re)initialization* once neglected now dominates runtime. At 96k GPUs, Si et al. (2025) show that NCCL.init and similar routines become a major bottleneck; even after optimization (Jiang et al., 2024; Si et al., 2025), key collectives still scale linearly with $\#GPU$. Hu et al. (2025) similarly report that large messages throughput in NCCL is best with linear-scaling algorithms. As a result, restart latency is projected to grow linearly with $\#GPU$ while MTBF shrinks inversely, pushing LLM pretraining system towards a *restart-dominant regime* where expected restart latency prevails over useful work time. For example, a 600k+ cluster of 5-min MTBF may spend ≥ 60 -min on global restart after every failure. To address this challenge, three major approaches have been considered to achieve fault-tolerance in distributed training: checkpointing, partial recovery, and replication.

Checkpointing attempts to minimize rework burden from progress loss by failures. Universal checkpointing (Lian et al., 2025) decouples checkpoint state from a fixed par-

¹Fraction of time the system is operational: uptime divided by total wall-clock time (Avizienis et al., 2004).

²Total wall-clock time of training. Job Completion Time.

allelism configuration to enable flexible and fast recovery. GEMINI (Wang et al., 2023b) maintains lightweight in-memory snapshots for near-instant rollback. Just-in-Time checkpointing (Gupta et al., 2024) adapts snapshot timing to high risk windows. Oobleck (Jang et al., 2023) reduces replay cost in pipeline parallelism (Huang et al., 2019) by replaying short segments. Multi-level checkpointing (Di et al., 2014; Maurya et al., 2024; 2026) amortizes I/O across device, host, and storage tiers. These advances lighten the rework burden *once the system is back on the job*. In the restart-dominant regime, however, the performance bottleneck shifts to restart downtime: the cumulative waste spent *before the system can resume the job*. In this setting, reducing the number of global restarts becomes critical to improving system availability hence reducing time-to-train.

In parallel, traditional replication methods have been revisited to improve availability by masking failures with redundant computation, and partial recovery has been proposed to replace global restarts with cheaper, local system recoveries (Losada et al., 2020). Ferreira et al. (2011) disclosed that even minimal redundancy $r = 2$ can tolerate a large number of failures and Benoit et al. (2019) proposed an optimized strategy to minimize time-to-train with checkpointing. However, replication is constrained by a fundamental ceiling: degree- r replication incurs $r \times$ more computation, which becomes prohibitive in practice. This strongly motivates the need for a direct yet practical approach to improving availability without extravagant waste, especially as we are at the doorstep of the restart-dominant regime.

We hereby propose SPARe: Stacked Parallelism with Adaptive Reordering, a failure-masking scheme as effective as traditional replication yet pays only near-constant overhead around $2 \sim 3 \times$ even for high redundancy like $r \sim 20$. Instead of fully replicating computation, SPARe stacks shards of computation across synchronous Data Parallelism (Zinkevich et al., 2010; Sergeev & Del Balso, 2018) so that redundant workload can be minimized by adaptively reordering the stacks. SPARe operates entirely on the data-parallel layer hence agnostic to any model architecture and inner parallelism topology. Our contributions in this work are:

- We derive working formulas for the expected failure count and computation overhead SPARe endures and pays, and validate them with our simulation results.
- We optimize SPARe with checkpointing (SPARe+CKPT) to find optimal redundancy r to minimize time-to-train.
- Lastly, we provide discrete-event simulations using the core components of FedDES (Chen et al., 2025), a large parallel system simulation toolkit built on top of SimGrid (Casanova et al., 2014). With realistic system parameters for a 600k H100 cluster, we benchmark SPARe+CKPT against replication+CKPT and CKPT-only, showing 40 $\sim$ 50% gain in time-to-train.

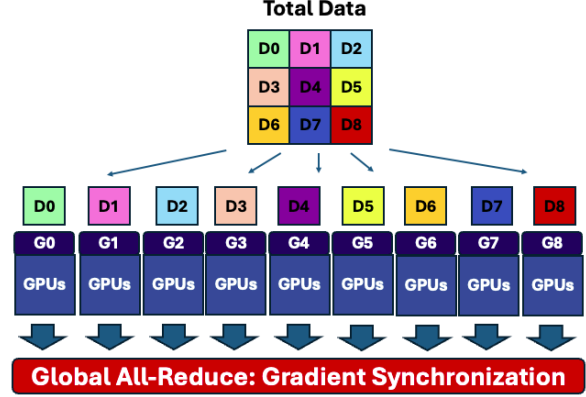


Figure 1. Synchronous Data Parallelism.

2. Background

2.1. Synchronous Data Parallelism

Fig. 1 shows synchronous Data Parallelism (Zinkevich et al., 2010; Sergeev & Del Balso, 2018) under a Megatron-style hybrid-parallel topology (Shoeybi et al., 2019). Let N be the data-parallel degree and M be the size of each model-parallel group. The system consists of N model-parallel groups each containing M GPUs and collectively holding one model replica. Each group may internally combine tensor, pipeline, sequence, or expert parallelism (Narayanan et al., 2021; Huang et al., 2019; Korthikanti et al., 2023; Fedus et al., 2022); in such settings, a single node failure typically interrupts the entire group (Salpekar et al., 2026). Throughout this paper, a *group* refers to one model-parallel group, equivalently one logical data-parallel replica.

This layout induces M data-parallel groups. For each local model-parallel rank $m \in \{0, \dots, M-1\}$, the same-rank GPUs across the groups form one data-parallel group $\{(0, m), (1, m), \dots, (N-1, m)\}$, synchronized by a communicator of world size N . Thus, the system has M data-parallel communicators in total. Following recent scaling trends (Dubey et al., 2024; Chu et al., 2025), M can be hundreds to a few thousands; we consider $N \sim 10^2-3$. See App. F.1 for the communicator topology figure.

At each training step, group i computes a *partial gradient* $\mathbf{g}_i$ from data shard D_i . The data-parallel communicators then all-reduce the logical *full gradient* $\bar{\mathbf{g}} = \frac{1}{N} \sum_{i=0}^{N-1} \mathbf{g}_i$ and distribute the corresponding gradient shards for the model update. Hence, all N partial gradients must be collectible; losing any group makes synchronization unavailable and interrupts vanilla Data Parallelism.

2.2. Node Failures in Large Parallel HPC System

In large GPU clusters, node failures occur when nodes are forcibly lost by fail-stop faults such as GPU memory error, driver/kernel faults, process crash, power/thermal ex-

cursions and etc., and the aggregate failure rate rises with $\#GPU$ (Schroeder & Gibson, 2009). Kokolis et al. (2025) empirically validated that MTBF decreases by $\propto \frac{1}{\#GPU}$ for jobs with more than 32 GPUs. As MTBF decreases, time wasted outside of training increases, hence the total wall-clock time to finish training also increases.

The well-known Young & Daly’s formula (Young, 1974; Daly, 2006) gives optimal checkpointing period that minimizes the time to finish training when the rework waste on lost progress is the primary bottleneck. However, in restart-dominant regime, system downtime is of main concern; in this setting minimizing the *portion* of the downtime, hence maximizing the system availability, leads to the minimal time to finish training. For system failure interval T_f , checkpointing period T_c , checkpoint save time T_s , and global restart cost T_r , recent work of Saxena et al. (2024) gives optimal checkpointing period for maximal availability as:

$$T_c^* = T_s + \sqrt{T_s^2 + 2T_s(T_f + T_r)}, \quad (1)$$

where corresponding maximal system availability is:

$$A^*(T_f, T_s, T_c^*, T_r) = \frac{T_f - \frac{T_f T_s}{T_c^*}}{T_f + \frac{T_c^*}{2} + T_r}. \quad (2)$$

In our work T_r, T_s are fixed parameters decided by model size and system capacity, while T_f is a design factor decided by redundancy r for given data-parallel degree N . Hence we denote T_c^* and A^* as functions of T_f : $T_c^*(T_f)$, $A^*(T_f)$.

2.3. Partial Recovery: Avoiding Global Restart

Partial recovery minimizes downtime by localizing recovery across surviving nodes so that system can resume job without going through the costly global restart (Bland et al., 2013). For example, (1) **communicator shrinking** excludes failed nodes (nodes) from the job and forms a new communicator over the survivors (Losada et al., 2020; ncc, 2025), (2) **Non-shrinking replacement** respawns failed nodes or activates hot spares to merge them back in (Losada et al., 2020), (3) **Rollback with message logging** restarts only the failed nodes from a checkpoint and uses logged/replayed messages to restore a consistent global state (Elnozahy et al., 2002), (4) **Runtime reinitialization** rebuilds runtime and communicator state through the batch system (Georgakoudis et al., 2020), and (5) **Task migration** dynamically remaps failed work units onto healthy resources (Chakravorty et al., 2006).

In this paper, we consider shrinking paired with replication or checkpointing as the default partial recovery, as it is fastest at re-establishing collective communication across surviving groups and also directly supported by NCCL (ncc, 2025), PyTorch (tor, 2024), and MPI (Losada et al., 2020). Moreover, Bland et al. (2015) report shrinking takes tens of milliseconds for hundreds of nodes. As the data-parallel

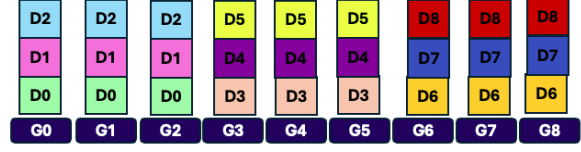


Figure 2. Traditional Replication $r = 3$.

communicators are of world size hundreds to a few thousands, we therefore treat their shrink costs as negligible: we presume the job resumes without any meaningful delay after each failure if partial recovery is available for the communicators as long as all partial gradients are collectible.

2.4. Traditional Replication: Robust yet Expensive

Fig. 2 shows replication of degree $r = 3$. We define a term, *type*, to denote shard identity: type i is the partial gradient contribution associated with data shard D_i . With partial recovery, system can endure multiple failures without job interruption as long as all types of shards are surviving on the job. However, assuming fixed GPU budget, each group now hosts $3 \times$ shards compared to the original Data Parallelism, hence $3 \times$ workload if we regard the data size as equivalent to the computation amount for simplicity. Assuming random independent failures, higher redundancy r masks more failures before depletion (Ferreira et al., 2011), yet also linearly inflates the workload by $r \times$, a prohibitive overhead that degrades time-to-train in practice.

3. SPARE: Stacked Parallelism with Adaptive Reordering

Now we present the SPARE framework. In the imminent restart-dominant regime, one way to improve the system availability (2) to a practical level ($\geq 90\%$) is to increase the system failure interval T_f , especially since reducing the global restart cost T_r below the linear scalability has been revealed to be a very subtle problem (Si et al., 2025; Hu et al., 2025). Traditional replication easily increases T_f by masking multiple node failures with redundant computation, yet the linearly inflating computation overhead is prohibitive in practice. SPARE aims to answer to this challenge:

Can we mask frequent failures with redundant computation while keeping the overhead down to be near-constant?

Sec. 3.1 introduces the key ideas of SPARE to solve this challenge, and Sec. 3.2 explains the algorithm flow of SPARE.

3.1. Key Ideas of SPARE

Traditional replication requires the groups to compute all assigned data to collect all partial gradients. Key intuition behind SPARE is to replicate *shards* of computation (data) instead of fully replicating each group, and stack them across

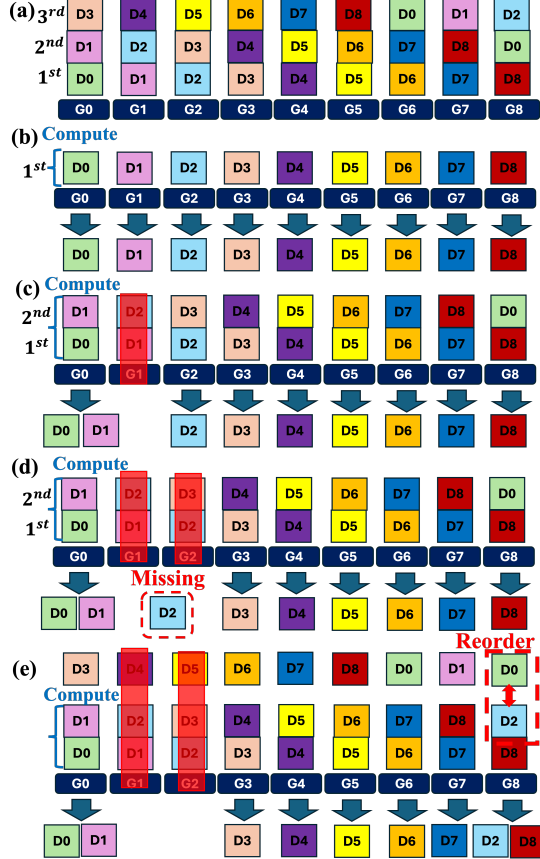


Figure 3. (a): Example of SPARE at $N = 9$, $r = 3$. (b): Before any failure, all partial gradients can be collected after computing the 1st stack. (c): With group 1 failure, system needs to compute up to 2nd stack to collect all types. (d): If group 2 fails later, type 2 partial gradient cannot be collected within the 2nd stack of shards. (e): However, all partial gradients can be collected after computing up to 2nd stack when group 8 stack is reordered.

parallelism so that all types of partial gradients are collectible *before* computing all assigned data (stacks) at each group. Below are the key steps of SPARE of redundancy r :

- From synchronous Data Parallelism introduced in Sec. 2.1, replicate all N types of data shards $\{D_0, D_1, \dots, D_{N-1}\}$ and stack them up to r stacks with cyclic rotation across parallelism so that all types are present in every stack.
- At each training step, schedule gradient synchronization right after all types of shards are computed: aggregate partial gradients as soon as all types are collectible.
- To compute all types of shards within the minimal number of stacks at each training step, reorder the shard stack in each group accordingly to the failure accumulation.

In this way, redundant computation needed to mask failures can be kept down to be minimal at each training step. Also, the adaptive reordering changes only the supplier of each

Algorithm 1 SPARE training loop

Require: active groups each with shard stacks

- 1: all groups active; all-reduce stack $\leftarrow 1$
- 2: **while** training **do**
- 3: **for** $j = 1$ **to** all-reduce stack **do**
- 4: each active group computes j^{th} stack
- 5: **end for**
- 6: all-reduce
- 7: **if** success **then**
- 8: update parameters; **continue**
- 9: **end if**
- 10: detect node failure(s) and failed group(s);
- 11: RECTLR detects system failure
- 12: **if** system failure **then**
- 13: global restart; all groups active; reset shard stacks; reset all-reduce stack $\leftarrow 1$; **continue**
- 14: **end if**
- 15: RECTLR decides if reordering is needed
- 16: **if** reordering needed **then**
- 17: find minimal all-reduce stack; reorder;
- 18: **end if**
- 19: compute shard(s) lost by failure(s) on current step;
- 20: communicator shrink; all-reduce; update parameters;
- 21: commit new all-reduce stack and reordered stacks; **continue**
- 22: **end while**

shard type, not the collected full gradient, hence leaves the optimizer state and the resulting update unchanged. With judicious scheduling of gradient synchronization and failure-adaptive reordering, SPARE needs to compute only $2 \sim 2.8$ stacks in average to collect all partial gradients at each training step even for high redundancies such as $r = 20$, whereas traditional replication requires to finish computing all $r = 20$ stacks amount of data in every training step. Operating entirely at the data-parallel layer, SPARE is agnostic to model architecture and inner parallelism topology.

3.2. Algorithm Flow of SPARE

Before we explain the algorithm flow of SPARE we first define the failure ontology we use throughout this work. We denote an individual GPU failure as a *node failure*. In Data Parallelism any node failure interrupts the job and enforces global restart. However, for replication and SPARE, global restart occurs only when any shard type i depletes with failure accumulation: we refer to this incident as the *wipe-out* of shard i , and also as *system failure*. Lastly, we assume the node failures are detected when the system calls all-reduce across data-parallel groups for gradient synchronization, following the typical large parallel system convention.

The training loop of SPARE develops on the synchronous Data Parallelism introduced in Sec. 2.1, except that it does

Algorithm 2 RECTLR algorithm**Require:** current shard stacks, current all-reduce stack**Ensure:** Either RESTART or updated stacks

```

1:  $S_0 \leftarrow$  all-reduce stack
2: Phase 0: decide if reordering is needed.
3: if HK-FIXED succeeds then
4:   return current stacks {no reordering}
5: end if
6: Phase 1: find minimal all-reduce stack.
7:  $S^* \leftarrow$  UNDEFINED
8: for  $S = S_0$  to  $r$  do
9:   if HK-FREE succeeds then
10:     $S^* \leftarrow S$ ; break
11:   end if
12: end for
13: if  $S^*$  is UNDEFINED then
14:   flag system failure to trigger global restart
15: end if
16: all-reduce stack  $\leftarrow S^*$ 
17: Phase 2: reorder with minimal movement.
18: run MCMF on current stacks and all-reduce stack
19: return updated all-reduce stack and reordered stacks

```

not compute all the stacks to complete a training step. Instead, it triggers global all-reduce as soon as a committed number of stacks are computed, which is aimed to be the minimal stacks required to collect all partial gradients so that the redundant computation can be minimized at each step. We denote this committed number as *all-reduce stack*, which is set as 1 by default when training starts.

When node failure happens, the next all-reduce will fail with collective hang/drop and system will detect the failure. SPARe then initiates the *reordering controller*, RECTLR, which serves three purposes:

- System failure detection;
- Finding the minimal all-reduce stack for subsequent steps;
- Reorder stacks accordingly with minimal movement.

If RECTLR detects system failure, system undergoes global restart, reset the stacks to the original order and the all-reduce stack back to be 1. If not, RECTLR finds the minimal all-reduce stack with the new failure(s) and reorder the stacks accordingly. After RECTLR, system needs to collect the missing partial gradients lost by the new failure(s) to complete the current training step, hence commands all surviving model-parallel groups hosting the missing shard types to compute them. We denote this additional computation stack as *patch compute*. After patch compute, the system shrinks the communicators across all

data-parallel groups, performs gradient synchronization and model updates, commits the new all-reduce stack and re-ordered stacks, then proceeds to the next training step. See Alg. (1) for the pseudo-code of the SPARe training loop.

RECTLR plays the central role in SPARe training loop and it runs with 3 phases:

- **Phase 0** decides if reordering is needed with the new failure(s), by checking if the current all-reduce stack can collect all partial gradients across data-parallel groups via Hopcroft-Karp (HK) algorithm (Hopcroft & Karp, 1973) on the *fixed* stacks of shards; we denote it as HK-FIXED. HK algorithm checks the bipartite graph feasibility between N shard types to the stacks of surviving model-parallel groups up to the all-reduce stack. If HK-FIXED succeeds, no reordering is needed, hence RECTLR quits.
- **Phase 1** searches for the minimal all-reduce stack that can collect all partial gradients, by incrementally iterating the HK algorithm from the previous all-reduce stack up to redundancy degree r , now allowing *free permutation* of shard stacks within each group (HK-FREE). If no HK-FREE iteration succeeds, a wipe-out has occurred, and RECTLR flags a system failure to trigger a global restart.
- **Phase 2** finds how to reorder the stacks with minimal movement to achieve the newly found minimal all-reduce stack, by running min-cost max-flow algorithm (Goldberg & Tarjan, 1990), MCMF.

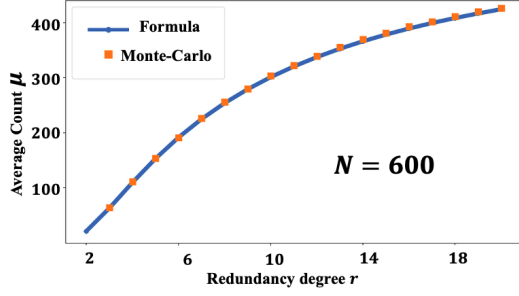
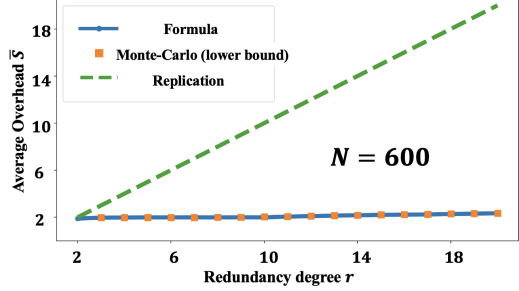
See Alg. (2) for the pseudo-code of RECTLR. At $N \sim 10^{2-3}$, HK-FIXED, HK-FREE, and MCMF do not require significant computational cost. See App. D for the detailed descriptions and complexity analysis of RECTLR.

4. Theoretical Analysis of SPARe

In this Section, we provide theoretical analysis on three key properties of SPARe:

- How many failures can SPARe endure before wipe-out?
- How much computation overhead SPARe needs to pay?
- What is the optimal redundancy for minimal time-to-train when merged with checkpointing?

We provide key results on the first two questions in Sec. 4.1 and show joint optimization with checkpointing in Sec. 4.2. In our theoretical analysis, we assume exponential distribution on node failures (uniform interval). However, we consider the realistic Weibull distribution (Weibull, 1951; Schroeder & Gibson, 2009; Wang et al., 2023a; Min et al., 2025) in our simulations in Sec. 5.


 Figure 4. Average endurable failure count by redundancy r .

 Figure 5. Average computation overhead by redundancy r .

4.1. Theoretical Results about SPARe

In order to minimize the impact of each individual node failure, we design SPARe so that no pair of different shard types overlap in more than one group as in Fig. 3. See App. B.1 for the shard distribution rule that achieves this goal. In this way, wipe-out incidents of different shard types are almost independent to each other, hence their statistics can be approximated to follow the Poisson distribution (Chen, 1975; Barbour & Eagleson, 1983). This leads to two key results:

- SPARe can asymptotically endure as many failures as traditional replication: Thm. 4.1.
- SPARe can achieve computation overhead close to its theoretical lower bound: Thm. 4.2.

Theorem 4.1 (Average Failure Count). *The average failure count $\mu(N, r)$ SPARe can mask before the first wipe-out is asymptotically:*

$$\mu(N, r) \approx \frac{\Gamma(1/r)}{r} N^{1-1/r}, \quad (3)$$

where Γ refers to the Gamma function in NIST (2025).

Proof. According to Barbour & Eagleson (1983), wipe-out incidents are almost independent to each other so that Poisson approximation is valid (Chen, 1975):

$$\begin{aligned} \mu(N, r) &\approx \sum_{k \geq 0} \exp\left(-N \left(\frac{k}{N}\right)^r\right) \approx N \int_0^\infty e^{-Nt^r} dt \\ &= \frac{1}{r} N^{1-1/r} \Gamma\left(\frac{1}{r}\right) = \frac{\Gamma(1/r)}{r} N^{1-1/r}. \end{aligned} \quad (4)$$

See App. B.2 for the full proof. $\square$

$\mu(N, r)$ derived in (3) is consistent to the traditional replication formula given by (Ferreira et al., 2011). Fig. 4 shows that SPARe can endure up to 426 failures in average for $N = 600$ with redundancy $r = 20$. This means that for a system of MTBF m , SPARe effectively increases it up to $426 \times m$, substantially improving the system availability. Traditional replication requires $20 \times$ computation overhead for this availability gain. However, with adaptive reordering, SPARe only pays $2.8 \times$ for the same availability gain.

Theorem 4.2 (Average Computation Overhead). *The average computation overhead $\bar{S}(N, r)$ SPARe needs to pay before the first wipe-out is approximately*

$$\bar{S}(N, r) \approx \frac{1}{\lfloor \mu \rfloor} \sum_{k=0}^{\lfloor \mu \rfloor - 1} (c(k) + \rho_k), \quad (5)$$

where μ is the average failure count of Thm. (4.1) and $c(k)$ is the lower-bound of all-reduce stack: $c(k) := \left\lceil \frac{N}{N-k} \right\rceil$. ρ_k is the probability of patch compute at k failures, $\rho_k := \max\{0, 2N - n_k\}/n_k$, where $n_k := c(k)(N - k)$.

$\bar{S}(N, r)$ has an idealistic lower bound of:

$$\bar{S}(N, r) \gtrsim \frac{1}{\lfloor \mu \rfloor} \sum_{k=0}^{\lfloor \mu \rfloor - 1} c(k), \quad (6)$$

achievable if system detects failures earlier than the global all-reduce hence does not require patch computes.

Proof. Probabilities of adversarial cases where all-reduce stack is forced to be $> c(k)$ can be formulated by Hall’s Marriage Theorem (Hall, 1987), which are negligible at $N \sim 10^{2-3}$. Therefore all-reduce stack is $\approx c(k)$. Expected number of patch compute per step can be approximated by noting that the probability of a random failure hitting any singleton type³ scales as ρ_k : $\Pr(\text{patch compute} \mid k) \approx \rho_k$. See App. B.3 for the full proof. $\square$

Fig. 5 clearly shows that the average computation overhead of SPARe is near-constant around $2 \sim 2.8 \times$ compared to the linearly scaling $r \times$ of traditional replication. As SPARe achieves as high availability as replication, near-constant overhead results in significant gain in time-to-train.

See App. C for Monte-Carlo validation on $\mu(N, r)$ and the lower bound of $\bar{S}(N, r)$, which shows 1.13% and 0.60% absolute error respectively.

Failure heterogeneity and correlations Real systems may exhibit heterogeneous failure rates across GPUs and correlated failures within the same allocation group or failure domain (Thorpe et al., 2023; Wan et al., 2025). These

³Type that is computed only once before all-reduce, hence its loss prevents gradient synchronization.

effects do not change the feasibility logic of SPARE: Alg. (1) and Alg. (2) operate only on the realized failure count and survivor set, and therefore do not require a specific temporal failure law. They may, however, affect the distribution of survivor sets and hence the accuracy of the closed-form averages in Thm. 4.1 and Thm. 4.2. In practice, this can be mitigated by making the shard placement independent of physical failure domains, such as spreading replicas across racks, zones, or parallelism groups, as commonly done in failure-aware training systems such as Bamboo (Thorpe et al., 2023) and ByteRobust (Wan et al., 2025).

4.2. Joint Optimization with Checkpointing

SPARE cannot mask failures indefinitely hence must be paired with checkpointing to ensure forward progress under continual system failures. Merging two schemes, SPARE+CKPT, entails two coupled trade-offs: one that hinges on checkpointing period, checkpointing overhead versus rework amount, and the other on redundancy r , availability gains versus computation overhead. In the following, we provide the joint optimization of SPARE+CKPT to find the optimal redundancy r and corresponding checkpointing period that minimize time-to-train.

Let T_0 be the time-to-train of N -way Data Parallelism in no failure scenario. Then the useful work time a SPARE needs to finish is $T_0 \times \bar{S}(N, r)$. Since system availability (2) is the fraction of useful work time over time-to-train, we can set a normalized time-to-train function $J(r)$ as:

$$J(r) := \frac{\text{time-to-train}}{T_0} = \frac{\bar{S}(N, r)}{A^*(\mu(N, r)m)}. \quad (7)$$

where m is the system MTBF on node failures.

Theorem 4.3 (Optimal r^* for minimal time-to-train). *Using checkpointing period of Eq. (1), SPARE+CKPT achieves minimal time-to-train at optimal redundancy r^* :*

$$r^* \approx \lceil \log_2 N + 0.833 \rceil. \quad (8)$$

Proof. Substitute Eq. (1) (2) (3) (5) to $J(r)$. $J(r)$ is minimized around r^* that satisfies $\mu(N, r^*) \approx N/2$ and $\bar{S}(N, r^*) \approx 2$. Let $\varepsilon := 1/r^*$ and use $\Gamma(\varepsilon) = \frac{1}{\varepsilon} - \gamma + O(\varepsilon)$. Solving it with logarithm yields

$$r^* \approx \log_2 N + \frac{\gamma}{\ln 2} \approx \log_2 N + 0.833. \quad (9)$$

See App. B.4 for the full proof. $\square$

5. System Performance Evaluation

This section evaluates the performance of SPARE using a discrete-event simulator implemented upon FedDES (Chen et al., 2025), a simulation toolkit developed on top of SimGrid (Casanova et al., 2013). SimGrid provides a mature and

Table 1. DES system parameters for 600k H100 cluster.

PARAMETER	SETTING
FAILURE	MTBF 300s, $k = 0.78$ (WEIBULL)
GLOBAL RESTART	$T_r = 3600$ s
MODEL SIZE	10T PARAMS (20TB)
DP GROUPS	$N \in \{200, 600, 1000\}$
DATA/STACK	256M TOKENS (4×64 M)
COMPUTE/STACK	$T_{\text{comp}} = 64$ s PER STACK
TRAIN HORIZON	$T_0 = 10,000 \text{ STEPS} \times (T_{\text{comp}} + T_a)$
ALL-REDUCE	$T_a = 2, 6, 10$ s AT EACH N
FAILED ALL-REDUCE	50% OF ALL-REDUCE, $0.5 \times T_a$
COMM. SHRINK	0.1s
CKPT TIME	$T_s = 60$ s
EVENT JITTER	$\times \mathcal{N}(1, 0.05^2)$ ON ALL EVENTS

validated simulation framework for modeling distributed systems and applications (Binkert et al., 2011; Varga, 2001; Casanova et al., 2008). In discrete-event simulation, system execution is represented as a chronology of time-stamped events, where time advances by repeatedly processing the next scheduled event rather than by stepping through a fixed time grid (Tocher & Owen, 2008; Neuwirth & Paul, 2021). This event-driven abstraction enables controlled and repeatable failure injection and realistic end-to-end training time accounting, even at system scales that are impractical to reproduce experimentally such as a 600k-H100 cluster (Casanova et al., 2014). For further details on our choice of simulator, see App. E. Our simulator compares SPARE+CKPT against two baselines: replication with checkpointing (Rep+CKPT) and standard data parallel training with checkpointing only (CKPT-only), under realistic large-scale system parameters.

5.1. Realistic System Parameters

Table 1 shows system parameters we have set to emulate a restart-dominant system of 600k H100s, where MTBF is projected to be $m = 5$ -min according to the scaling law validated by Kokolis et al. (2025) and MTBF reported for 16k H100s (Dubey et al., 2024). We set the global restart to take $T_r = 60$ -min to test SPARE and baselines on a harsh restart-dominant regime. We set the LLM to be of 10T parameters of memory size 20 TB (FP16). We consider three cases of data-parallel degree: (i) $N = 200$, (ii) $N = 600$, and (iii) $N = 1000$ parallel groups. From the Llama 3 training report of Dubey et al. (2024) and Chu et al. (2025), we project the compute power per GPU to be 400 TFLOPs and one shard of training data to be of size 4×64 M Tokens, considering 4 gradient accumulations. Consequently, we set the compute time per shard to be $T_{\text{comp}} = 64$ s⁴. We model the global all-reduce for gradient synchronization to be ring-based (Patarasuk & Yuan, 2009), hence scales up linearly by N and take $T_a = 2, 6, 10$ s for each N considering the partial gradient size of 20TB and conservative collective

⁴Regardless of N , per GPU workload is the same $\frac{256\text{M Tokens}}{400\text{TFLOPs}}$.

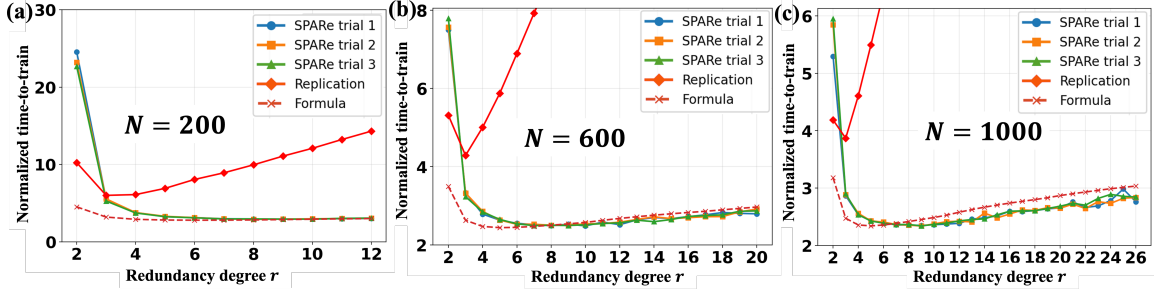


Figure 6. Time-to-train/ T_0 of SPARE+CKPT / Rep+CKPT from simulation and $J(r)$ (7) for (a) $N = 200$, (b) $N = 600$, (c) $N = 1000$.

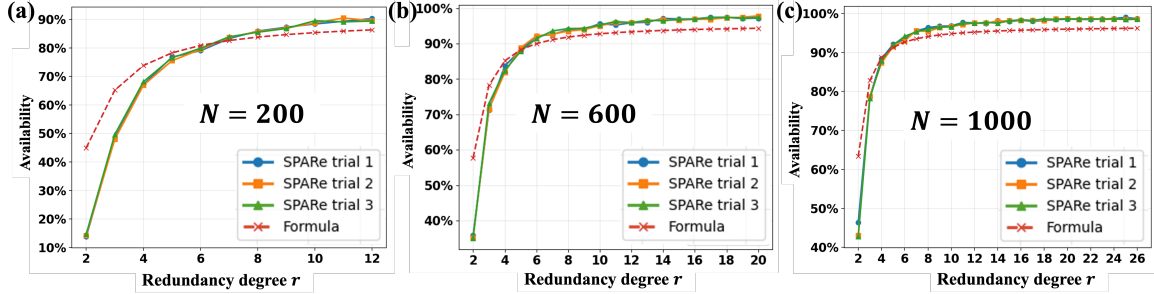


Figure 7. Availability of SPARE+CKPT from simulation and $A^*(\mu m)$ (2) for (a) $N = 200$, (b) $N = 600$, (c) $N = 1000$

goodput per GPU of 400 Gb/s. Therefore, we set a training step that computes one data shard to take ($T_{\text{comp}} + T_a$) for simplicity, and set 10,000 steps to finish the training: the base time-to-train in the no-failure scenario is $T_0 = 10,000 \times (T_{\text{comp}} + T_a)$.

We inject failure events following the realistic Weibull distribution (Weibull, 1951) and choose the seminal shape parameter of $k = 0.78$ as provided by Schroeder & Gibson (2009). Weibull distribution is widely recognized as a legitimate standard proxy to represent real-life failures at the group-level abstraction as shown in Wang et al. (2023a) and the Titan trace study (Min et al., 2025). As we model a system that detects failed group(s) only at the global all-reduce step, we assume the *failed* all-reduce to take, in expectation, half the time of a successful one, without loss of generality.

Lastly, we set both shrink cost and reordering controller cost to be 0.1s. For the data-parallel degree of N , each data-parallel group consists of N GPUs hence the synchronizing communicator is of world size N . Therefore, our shrink cost setting is justified by the empirical study of Bland et al. (2015) for given $N = 200, 600, 1000$. Furthermore, checkpoint save time is set to be 1-min and optimal checkpointing period T_c^* is calculated from Eq. (1). Lastly, compute jitter of normal distribution $\times \mathcal{N}(1, 0.05^2)$ is added to all events to reflect real-life variances and system noise.

5.2. Performance Evaluation Results

We created three event trails and ran each for three schemes of SPARE+CKPT, Rep+CKPT, and CKPT-only.

See App. F.2 for the flowchart of each scheme.

5.2.1. RESULTS OF CONVENTIONAL BASELINES

Under the harsh restart-dominant setting, CKPT-only did not proceed more than a few steps in a time where other schemes would finish the training. On the other hand, Rep+CKPT achieved minimal time-to-train at low redundancy $r = 3$ in all cases as shown in Fig. 6, consistent to prior works of Ferreira et al. (2011) and Elliott et al. (2012). For higher redundancies however, Fig. 6 clearly shows that the time-to-train of Rep+CKPT inflates linearly from the $r \times$ overhead.

5.2.2. SPARE RESULTS AND GAIN ANALYSIS

SPARE+CKPT proves to fully exploit the significant availability ($> 90\%$) at high redundancies by keeping the computation overhead low ($2 \sim 3 \times$) with failure-adaptive reordering. Fig. 6 shows the normalized time-to-train $J(r) = \text{time-to-train}/T_0$ along with its theoretical prediction from $J(r)$ (7), and Fig. 7 shows empirical availability with theoretical projection $A^*(\mu m)$ (2). SPARE+CKPT performs better than predicted at high redundancies, showing higher availability and clearly lower time-to-train. That is because failure rate effectively decreases for high accumulation, as it is proportional to the number of active GPUs (Schroeder & Gibson, 2009; Kokolis et al., 2025); as high $r \rightarrow$ high $\mu(N, r)$, hence for higher r and N failure masking scheme is more advantageous. Fig. 8 shows the average number of stacks computed per training step, in other words, the empirical computation overhead along

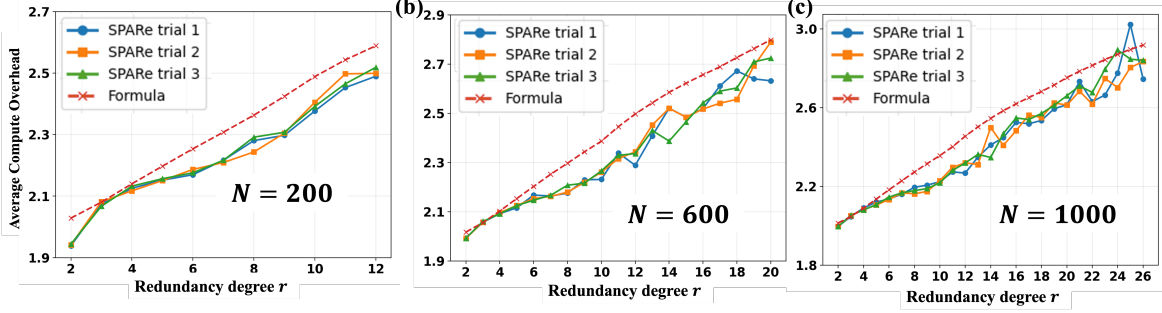


Figure 8. Average Computation Overhead of SPARe from simulation and $\bar{S}(N, r)$ (5) for (a) $N = 200$, (b) $N = 600$, (c) $N = 1000$

with its theoretical prediction of $\bar{S}(N, r)$ (5). Our formula closely fits the empirical values within $\leq 4\%$ absolute error, validating Thm. (4.2).

However, SPARe+CKPT performs worse than expected at low redundancies, even worse than Rep+CKPT at $r = 2$. Fig. 8 explicitly shows that the theoretical prediction $\bar{S}$ (5) still closely fits the simulations at low redundancies, indicating our theories on computation overhead are solid. Therefore, the deterioration comes from the $k < 1$ Weibull distribution, where failure accumulates faster at early accumulation: low $r \rightarrow$ low $\mu(N, r)$, hence system encounters the disruptive global restarts much more frequently than our predictions made upon exponential distribution. This implies that this issue does not set a fundamental limit on SPARe, and can be easily improved by employing dynamic-based checkpointing (Bougeret et al., 2011; Benoit et al., 2022) effective for Weibull distribution. We

Table 2. Minimum time-to-train averaged over 3 trials.

N	Rep+CKPT		SPARe+CKPT		Gain
	time-to-train/ T_0	Availability	time-to-train/ T_0	r^*	
200	6.07	61.74%	2.92	9	87.00%
600	4.27	79.89%	2.49	8	93.90%
1000	3.88	84.41%	2.34	9	96.54%
					51.9%
					41.7%
					39.6%

evaluate the gain of SPARe+CKPT versus Rep+CKPT on the minimal time-to-train/ T_0 achieved in each scheme and N (Table 2). Compared to the best of Rep+CKPT, the best of SPARe+CKPT achieved 40 ~ 50% gain in time-to-train. Note that the theoretical predictions of the optimal r^* (8) for SPARe+CKPT are $r^* = 8, 10, 10$ at $N = 200, 600, 1000$ in Thm. (4.3), showing slight discrepancies coming from the exponential distribution assumption: for low N use SPARe with higher redundancy than r^* (8), and for high N use lower redundancy than r^* (8).

6. Related Works

Recent fault-tolerance systems for large-scale training reduce recovery cost after failures through storage hierarchy, hot spares, or topology-aware reconfiguration. Storage-assisted checkpointing systems, such as GEMINI (Wang

et al., 2023b) and DataStates-LLM (Maurya et al., 2024; 2026), keep recoverable training state in memory or storage tiers to reduce rollback and checkpoint I/O cost; SPARe is orthogonal and complementary, as it reduces the *frequency* of rollback recovery rather than the cost of executing it. Hot-spare and migration methods, such as TrainMover (Lao et al., 2024), reduce interruption time by preparing standby resources and migrating or replacing failed workers; in contrast, SPARe does not replace failed groups during tolerable failures, but shrinks the data-parallel communicators and lets surviving groups take over missing workloads by computing pre-stacked redundant data shards. Topology-aware recovery systems, including Bamboo (Thorpe et al., 2023), ReCycle (Gandhi et al., 2024), and FT-HSDP (Salpekar et al., 2026), exploit specific pipeline, data, or hybrid-parallel layouts to reroute work, adapt schedules, or recover failed training replicas. SPARe operates at a different abstraction layer: it sits above the inner model-parallel topology and only requires every partial-gradient type to remain collectible across surviving model-parallel groups. Thus, these systems can serve as inner-topology recovery substrates, while SPARe masks accumulated group failures without per-failure global restart or full-stack rescheduling.

7. Conclusion

In this work, we have introduced SPARe: Stacked Parallelism with Adaptive Reordering, which masks failures as many as traditional replication yet keeps the computation overhead as low as $2 \sim 3\times$ even for high redundancies. SPARe achieves 40 ~ 50% lower time-to-train compared to traditional replication baseline according to our realistic discrete-event simulations, at the harsh restart-dominant system setting projected for 600k H100 GPUs. In the imminent restart-dominant regime, masking failures, hence bypassing global restarts, is the most direct and intuitive strategy. SPARe positions itself as an effective and practical proposal for fault-tolerant LLM pretraining systems with 100k+ GPUs, highly versatile as it is agnostic to any model architecture and inner parallelism topology.

Impact Statement

This work aims to advance fault-tolerant distributed training system on to the imminent restart-dominant regime, with the goal of achieving system availability over 90% yet keeping computation overhead low to finish the training in practical time span. By substantially reducing the time-to-train required for next generation LLM pretraining, the proposed method of SPARe lowers the cost of frontier foundation model training and further elevate the ceiling for LLM scaling. More broadly, finishing foundation model training faster can contribute to lower energy consumption and more sustainable use of extreme scale HPC clusters. At the same time, increased accessibility to scale up LLM pretraining may accelerate progress of foundation model studies and development, impacting wide range of science and engineering. All experiments in this paper are conducted using publicly available libraries. The method itself is a fault-tolerance protocol for large parallel systems and does not introduce new capabilities that are inherently harmful. We do not foresee any immediate negative societal impacts directly resulting from this work.

Reproducibility Statement

Codes required to reproduce the experiments are provided in <https://github.com/padsysl/SPARe.git>, along with the simulation trajectories used in this work. See the flowcharts of each scheme in App. F.2.

Acknowledgements

We thank Ziyue Liu and Zhengyang Wang for helpful early discussions. This work was supported by the U.S. Department of Energy, Office of Science, Advanced Scientific Computing Research (ASCR), under contract DE-AC02-06CH11357, DE-SC0024207, and DE-SC0025390. This work was also supported by the National Science Foundation under contract CCF-2107321 and OAC-2623546.

References

- Torchft: Fault tolerance for PyTorch. <https://github.com/meta-pytorch/torchft>, 2024.
- NCCL Documentation: Shrinking a Communicator (ncclCommShrink). <https://developer.nvidia.com/blog/enabling-fast-inference-and-resilient-training-with-nccl-2-27/>, 2025.
- Avizienis, A., Laprie, J.-C., Randell, B., and Landwehr, C. Basic concepts and taxonomy of dependable and secure computing. *IEEE transactions on dependable and secure computing*, 1(1):11–33, 2004.
- Barbour, A. and Eagleson, G. Poisson approximation for some statistics based on exchangeable trials. *Advances in applied probability*, 15(3):585–600, 1983.
- Beaumont, O. and Rejeb, H. On the importance of bandwidth control mechanisms for scheduling on large scale heterogeneous platforms. In *2010 IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*, pp. 1–12. IEEE, 2010.
- Benoit, A., Herault, T., Fèvre, V. L., and Robert, Y. Replication is more efficient than you think. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–14, 2019.
- Benoit, A., Robert, Y., and Vivien, F. Checkpointing strategies to protect parallel jobs from non-memoryless fail-stop errors. In *IEEE International Conference on Cluster Computing (CLUSTER)*, San Diego, United States, September 2022. doi: 10.1109/CLUSTER51413.2022.00030. URL <https://inria.hal.science/hal-03610883>.
- Binkert, N., Beckmann, B., Black, G., Reinhardt, S. K., Saidi, A., Basu, A., Hestness, J., Hower, D. R., Krishna, T., Sardashti, S., et al. The gem5 simulator. *ACM SIGARCH computer architecture news*, 39(2):1–7, 2011.
- Bland, W., Bouteiller, A., Herault, T., Bosilca, G., and Donarra, J. Post-failure recovery of mpi communication capability: Design and rationale. *The International Journal of High Performance Computing Applications*, 27(3): 244–254, 2013.
- Bland, W., Lu, H., Seo, S., and Balaji, P. Lessons learned implementing user-level failure mitigation in mpich. In *2015 15th IEEE/ACM international symposium on cluster, cloud and grid computing*, pp. 1123–1126. IEEE, 2015.
- Bougeret, M., Casanova, H., Rabie, M., Robert, Y., and Vivien, F. Checkpointing strategies for parallel jobs. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–11, 2011.
- Carastan-Santos, D. and De Camargo, R. Y. Obtaining dynamic scheduling policies with simulation and machine learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–13, 2017.
- Carastan-Santos, D., De Camargo, R. Y., Trystram, D., and Zrigui, S. One can only gain by replacing easy backfilling: A simple scheduling policies case study. In *2019 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, pp. 1–10. IEEE, 2019.

- Casanova, H., Legrand, A., and Quinson, M. Simgrid: A generic framework for large-scale distributed experiments. In *Tenth International Conference on Computer Modeling and Simulation (uksim 2008)*, pp. 126–131. IEEE, 2008.
- Casanova, H., Giersch, A., Legrand, A., Quinson, M., and Suter, F. Simgrid: a sustained effort for the versatile simulation of large scale distributed systems. *arXiv preprint arXiv:1309.1630*, 2013.
- Casanova, H., Giersch, A., Legrand, A., Quinson, M., and Suter, F. Versatile, scalable, and accurate simulation of distributed applications and platforms. *Journal of Parallel and Distributed Computing*, 74(10):2899–2917, 2014.
- Chakravorty, S., Mendes, C. L., and Kalé, L. V. Proactive fault tolerance in mpi applications via task migration. In *International Conference on High-Performance Computing*, pp. 485–496. Springer, 2006.
- Chen, L. H. Poisson approximation for dependent trials. *The Annals of Probability*, 3(3):534–545, 1975.
- Chen, Z., Chen, W., Zhang, D., Kim, K., Li, G., Di, S., and Lu, X. Feddes: Discrete event based performance simulation for federated learning systems. In *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing*, pp. 1–16, 2025.
- Chernoff, H. A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *The Annals of Mathematical Statistics*, pp. 493–507, 1952.
- Chu, W., Xie, X., Yu, J., Wang, J., Phanishayee, A., Tang, C., Hao, Y., Huang, J., Ozdal, M., Wang, J., et al. Scaling llama 3 training with efficient parallelism strategies. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pp. 1703–1716, 2025.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to algorithms*. MIT press, 2022.
- Daly, J. T. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future generation computer systems*, 22(3):303–312, 2006.
- Degomme, A., Legrand, A., Markomanolis, G. S., Quinson, M., Stillwell, M., and Suter, F. Simulating mpi applications: the smpi approach. *IEEE Transactions on Parallel and Distributed Systems*, 28(8):2387–2400, 2017.
- Di, S., Bouguerra, M. S., Bautista-Gomez, L., and Cappello, F. Optimization of multi-level checkpoint model for large scale hpc applications. In *2014 IEEE 28th international parallel and distributed processing symposium*, pp. 1181–1190. IEEE, 2014.
- Donassolo, B., Casanova, H., Legrand, A., and Velho, P. Fast and scalable simulation of volunteer computing systems using simgrid. In *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*, pp. 605–612, 2010.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The LLaMA 3 herd of models. *arXiv e-prints*, pp. arXiv–2407, 2024.
- Elliott, J., Kharbas, K., Fiala, D., Mueller, F., Ferreira, K., and Engelmann, C. Combining partial redundancy and checkpointing for hpc. In *2012 IEEE 32nd International Conference on Distributed Computing Systems*, pp. 615–626. IEEE, 2012.
- Elnozahy, E. N., Alvisi, L., Wang, Y.-M., and Johnson, D. B. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys (CSUR)*, 34(3):375–408, 2002.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Ferreira, K., Stearley, J., Laros III, J. H., Oldfield, R., Pedretti, K., Brightwell, R., Riesen, R., Bridges, P. G., and Arnold, D. Evaluating the viability of process replication reliability for exascale systems. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–12, 2011.
- Gandhi, S., Zhao, M., Skiadopoulos, A., and Kozyrakis, C. Recycle: Resilient training of large dnns using pipeline adaptation. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pp. 211–228, 2024.
- Georgakoudis, G., Guo, L., and Laguna, I. Reinit: Evaluating the performance of global-restart recovery methods for mpi fault tolerance. In *International Conference on High Performance Computing*, pp. 536–554. Springer, 2020.
- Goldberg, A. V. and Tarjan, R. E. Finding minimum-cost circulations by successive approximation. *Mathematics of Operations Research*, 15(3):430–466, 1990.
- Golomb, S. W. *Shift register sequences: secure and limited-access code generators, efficiency code generators, prescribed property generators, mathematical models*. World Scientific, 2017.
- Gupta, T., Krishnan, S., Kumar, R., Vijeev, A., Gulavani, B., Kwatra, N., Ramjee, R., and Sivathanu, M. Just-in-time checkpointing: Low cost error recovery from

- deep learning training failures. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pp. 1110–1125, 2024.
- Hall, P. On representatives of subsets. In *Classic Papers in Combinatorics*, pp. 58–62. Springer, 1987.
- Hoeffding, W. Probability inequalities for sums of bounded random variables. *Journal of the American statistical association*, 58(301):13–30, 1963.
- Hopcroft, J. E. and Karp, R. M. An $n^2/2$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on computing*, 2(4):225–231, 1973.
- Hu, Z., Shen, S., Bonato, T., Jeaugey, S., Alexander, C., Spada, E., Dinan, J., Hammond, J., and Hoefler, T. Demystifying nccl: An in-depth analysis of gpu communication protocols and algorithms. In *2025 IEEE Symposium on High-Performance Interconnects (HOTI)*, pp. 48–59. IEEE, 2025.
- Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M., Lee, H., Ngiam, J., Le, Q. V., Wu, Y., et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- Isaev, M., McDonald, N., Dennison, L., and Vuduc, R. Calculon: a methodology and tool for high-level co-design of systems and large language models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–14, 2023.
- Jang, I., Yang, Z., Zhang, Z., Jin, X., and Chowdhury, M. Oobleck: Resilient distributed training of large models using pipeline templates. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 382–395, 2023.
- Jayaram Subramanya, S., Arfeen, D., Lin, S., Qiao, A., Jia, Z., and Ganger, G. R. Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 642–657, 2023.
- Jiang, Z., Lin, H., Zhong, Y., Huang, Q., Chen, Y., Zhang, Z., Peng, Y., Li, X., Xie, C., Nong, S., et al. {MegaScale}: Scaling large language model training to more than 10,000 {GPUs}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pp. 745–760, 2024.
- Jones, H. W. A method and model to predict initial failure rates. In *2020 Annual Reliability and Maintainability Symposium (RAMS)*, pp. 1–6. IEEE, 2020.
- Kokolis, A., Kuchnik, M., Hoffman, J., Kumar, A., Malani, P., Ma, F., DeVito, Z., Sengupta, S., Saladi, K., and Wu, C.-J. Revisiting reliability in large-scale machine learning research clusters. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 1259–1274. IEEE, 2025.
- Korthikanti, V. A., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoeybi, M., and Catanzaro, B. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems*, 5:341–353, 2023.
- Lao, C., Yu, M., Akella, A., Cao, J., Guan, Y., Zhang, P., Zheng, Z., Xu, Y., Zhai, E., Cai, D., et al. Trainmover: An interruption-resilient and reliable ml training runtime. *arXiv preprint arXiv:2412.12636*, 2024.
- Lian, X., Jacobs, S. A., Kurilenko, L., Tanaka, M., Bekman, S., Ruwase, O., and Zhang, M. Universal checkpointing: A flexible and efficient distributed checkpointing system for {Large-Scale}{DNN} training with reconfigurable parallelism. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pp. 1519–1534, 2025.
- Losada, N., González, P., Martín, M. J., Bosilca, G., Bouteiller, A., and Teranishi, K. Fault tolerance of mpi applications in exascale systems: The ulfm solution. *Future Generation Computer Systems*, 106:467–481, 2020.
- Maurya, A., Underwood, R., Rafique, M. M., Cappello, F., and Nicolae, B. Datastates-llm: Lazy asynchronous checkpointing for large language models. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, pp. 227–239, 2024.
- Maurya, A., Rafique, M. M., Cappello, F., and Nicolae, B. Datastates-llm: Scalable checkpointing for transformer models using composable state providers. *arXiv preprint arXiv:2601.16956*, 2026.
- Min, J., Hong, Y., Meeker, W. Q., and Ostrouchov, G. A spatially correlated competing risks time-to-event model for supercomputer gpu failure data. *Technometrics*, 67(3):531–545, 2025.
- Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., et al. Efficient large-scale language model training on gpu clusters using megatron-llm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, pp. 1–15, 2021.
- Neuwirth, S. and Paul, A. K. Parallel i/o evaluation techniques and emerging hpc workloads: A perspective. In

- 2021 IEEE International Conference on Cluster Computing (CLUSTER), pp. 671–679. IEEE, 2021.
- NIST. NIST Digital Library of Mathematical Functions. <https://dlmf.nist.gov/>, 2025.
- Ostrouchov, G., Maxwell, D., Ashraf, R. A., Engelmann, C., Shankar, M., and Rogers, J. H. GPU lifetimes on titan supercomputer: Survival analysis and reliability. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–14, 2020.
- Patarasuk, P. and Yuan, X. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2):117–124, 2009.
- Rashidi, S., Sridharan, S., Srinivasan, S., and Krishna, T. Astra-sim: Enabling sw/hw co-design exploration for distributed dl training platforms. In *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pp. 81–92. IEEE, 2020.
- Salpekar, O., Varma, R., Yu, K., Ivanov, V., Wang, Y., Sharif, A., Si, M., Xu, S., Tian, F., Zheng, S., et al. Training llms with fault tolerant hsdp on 100,000 gpus. *arXiv preprint arXiv:2602.00277*, 2026.
- Saxena, N. R., Hukerikar, S., Blaz, M., and Raj, S. Optimal checkpoint interval with availability as an objective function. *arXiv preprint arXiv:2410.18124*, 2024.
- Schroeder, B. and Gibson, G. A. A large-scale study of failures in high-performance computing systems. *IEEE transactions on Dependable and Secure Computing*, 7(4): 337–350, 2009.
- Sergeev, A. and Del Balso, M. Horovod: fast and easy distributed deep learning in tensorflow. *arXiv preprint arXiv:1802.05799*, 2018.
- Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Si, M., Balaji, P., Chen, Y., Chu, C.-H., Gangidi, A., Hasan, S., Iyengar, S., Johnson, D., Liu, B., Ren, R., et al. Collective communication for 100k+ gpus. *arXiv preprint arXiv:2510.20171*, 2025.
- Suter, F. A versatile simulated data transport layer for in situ workflows performance evaluation. In *2025 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 1–11. IEEE, 2025.
- Thorpe, J., Zhao, P., Eyolfson, J., Qiao, Y., Jia, Z., Zhang, M., Netravali, R., and Xu, G. H. Bamboo: Making pre-emptible instances resilient for affordable training of large {DNNs}. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pp. 497–513, 2023.
- Tocher, K. and Owen, D. The automatic programming of simulations. *Journal of Simulation*, 2(3):143–152, 2008.
- Varga, A. Discrete event simulation system. In *Proc. of the European Simulation Multiconference (ESM’2001)*, volume 17, 2001.
- Velho, P., Schnorr, L. M., Casanova, H., and Legrand, A. On the validity of flow-level tcp network models for grid and cloud simulations. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, 23(4):1–26, 2013.
- Wan, B., Liu, G., Song, Z., Wang, J., Zhang, Y., Sheng, G., Wang, S., Wei, H., Wang, C., Lou, W., et al. Robust llm training infrastructure at bytedance. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pp. 186–203, 2025.
- Wang, Y., Kang, X., Shi, S., He, X., Tang, Z., Pan, X., Zheng, Y., Wu, X., Zhou, A. C., He, B., et al. Fault-tolerant hybrid-parallel training at scale with reliable and efficient in-memory checkpointing. *arXiv preprint arXiv:2310.12670*, 2023a.
- Wang, Z., Jia, Z., Zheng, S., Zhang, Z., Fu, X., Ng, T. E., and Wang, Y. Gemini: Fast failure recovery in distributed training with in-memory checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 364–381, 2023b.
- Weibull, W. A statistical distribution function of wide applicability. *Journal of applied mechanics*, 1951.
- Won, W., Heo, T., Rashidi, S., Sridharan, S., Srinivasan, S., and Krishna, T. Astra-sim2. 0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pp. 283–294. IEEE, 2023.
- Young, J. W. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9): 530–531, 1974.
- Zinkevich, M., Weimer, M., Li, L., and Smola, A. Parallelized stochastic gradient descent. *Advances in neural information processing systems*, 23, 2010.

A. Notation

Table 3. Notation Table.

SYMBOL	MEANING
N	NUMBER OF DP GROUPS
r	REDUNDANCY DEGREE
G_r^N	OPTIMAL GOLOMB-RULER: LENGTH r MODULO N
$H_i \subset [N]$	HOST SET: INDICES OF GROUPS THAT HOST SHARD TYPE i .
$T_w \subset [N]$	TYPE SET: INDICES OF SHARD TYPES HOSTED BY GROUP w (SO $ T_w = r$).
$U_k \subset [N]$	SURVIVOR SET: INDICES OF ACTIVE GROUPS AFTER k FAILURES; $ U_k = N - k$.
$STK[w]$	PERSISTENT LOCAL STACK ORDER AT GROUP w : A PERMUTATION OF T_w USED AS ITS COMPUTE SEQUENCE.
$STK[w][j]$	SHARD TYPE COMPUTED AT THE j^{th} STACK OF GROUP w .
S_A	ALL-REDUCE STACK: SYSTEM TRIGGERS ALL-REDUCE AFTER ALL ACTIVE GROUPS FINISH S_A STACKS.
$S(U_k)$	MINIMAL FEASIBLE STACK SO THAT ALL SHARD TYPES ARE COLLECTIBLE ACROSS SURVIVORS U_k .
$c(k)$	CAPACITY LOWER BOUND FOR $S(U_k)$: $c(k) := \left\lceil \frac{N}{N-k} \right\rceil$, HENCE $S(U_k) \geq c(k)$.
F	FAILURE COUNT TO FIRST WIPE-OUT: $F := \min\{k \geq r : \exists i \in [N] \text{ s.t. } H_i \cap U_k = \emptyset\}$.
P_k	NO WIPE-OUT PROBABILITY AT FAILURE COUNT k : $P_k := \Pr\{F > k\}$.
$\mu(N, r)$	AVERAGE ENDURABLE FAILURE COUNT: $\mu(N, r) := \mathbb{E}[F]$.
q	PATCH-COMPUTE COUNT PER STEP.
T_f	MEAN TIME BETWEEN SYSTEM FAILURE (GLOBAL RESTART)
T_s	CKPT SAVE TIME
T_c	CKPT PERIOD
T_r	GLOBAL RESTART LATENCY

B. Proofs

B.1. Mathematical definition of SPARe

Definition B.1 (Cyclic Golomb Ruler distribution rule). Set $G_r^N = \{g_0, \dots, g_{r-1}\} \subset \mathbb{Z}_N^5$ as the optimal Golomb Ruler (Golomb, 2017) of length r , the set of unique pairwise differences modulo N with $g_0 = 0$ and minimal g_{r-1} . In other words, the pairwise difference set $\{\pm(g_a - g_b) \bmod N : 0 \leq a < b \leq r-1\}$ has $r(r-1)$ distinct non-zero elements. A SPARe scheme (N, r) distributes shards of type $i \in [N]$ across N groups with redundancy r as:

$$H_i := \{(i - g) \bmod N : g \in G_r^N\}, \quad (10)$$

where H_i is the *host set*, set of group indices hosting shard i , with the caveat of $N \geq (2g_{r-1} - 1)$. Equivalently, each

⁵ $\mathbb{Z}_N := \mathbb{Z}/N\mathbb{Z}$, the additive cyclic group of integers modulo N , i.e., residues $\{0, 1, \dots, N-1\}$ with addition mod N

group w hosts *type set*, set of shard indices group w hosts:

$$T_w := \{(w + g) \bmod N : g \in G_r^N\}. \quad (11)$$

We provide a lemma to prove that the cyclic Golomb Ruler distribution rule of Def. (B.1) achieves our intention.

Lemma B.2. *Following the cyclic Golomb Ruler distribution rule of Def. (B.1), any two distinct shard types of (N, r) share at most one host: $|H_i \cap H_j| \leq 1 (i \neq j)$.*

Proof. Assume $i \neq j$ share two hosts w_1, w_2 ($|H_i \cap H_j| \geq 2$). Then there would be distinct $g_a, g_b, g_c, g_d \in G_r^N$ which suffice $w_1 \equiv i - g_a \equiv j - g_b$ and $w_2 \equiv i - g_c \equiv j - g_d \pmod{N}$ from Eq. (10). That results as $g_a - g_b \equiv g_c - g_d \pmod{N}$ which contradicts to definition of G_r^N . $\square$

B.2. Proof of Theorem 4.1

Proof. Let F be the failure count at first wipe-out and $P_k := \Pr\{F > k\}$. Expanding $\Pr\{F = k\} = P_{k-1} - P_k$, expectation value of F can be written as:

$$\mu(N, r) := \mathbb{E}[F] = \sum_{k=0}^{N-1} P_k. \quad (12)$$

Fix failure count $k \in \{0, 1, \dots, N\}$. Conditional on exactly k node failures, the failed set $B_k := U_k^c \subset [N]$ is a uniformly random k -subset of $[N]$. For each group $w \in [N]$, define the exchangeable trials

$$Y_w := \mathbf{1}\{w \in B_k\} \in \{0, 1\}. \quad (13)$$

For each shard type $i \in [N]$, define the wipe-out event and indicator

$$A_i^{(k)} := \{H_i \subseteq B_k\}, \quad X_i^{(k)} := \mathbf{1}\{A_i^{(k)}\}. \quad (14)$$

Let the number of wiped-out types after k failures as:

$$W_k := \sum_{i \in [N]} X_i^{(k)} \quad (15)$$

be the number of wiped-out types after k failures. Then

$$P_k = \Pr\{F > k\} = \Pr\{W_k = 0\}. \quad (16)$$

Now, we use Barbour & Eagleson (1983, Thm. 1) on W_k to show wipe-out of different shard types are almost independent to each other, hence the statistics of W_k closely follows the Poisson approximation from Chen (1975): following the notations from Barbour & Eagleson (1983), the trials are $\{Y_w\}_{w \in [N]}$ (exchangeable), the subset size is r , and the index family is $\mathcal{N} := \{H_i : i \in [N]\}$ (so $|\mathcal{N}| = N$).

Wipe-out probability of type i at k failures can be written as:

$$p_k := \mathbb{E}[X_i^{(k)}] = \Pr(H_i \subseteq B_k) = \frac{\binom{N-r}{k-r}}{\binom{N}{k}} = \frac{(k)_r}{(N)_r}, \quad (17)$$

where $(x)_r := x(x-1) \cdots (x-r+1)$. Hence the expectation value of the number of types wiped-out at k failures is:

$$\lambda_k := \mathbb{E}[W_k] = Np_k. \quad (18)$$

For $i \neq j$, write $t_{ij} := |H_i \cap H_j|$. By Lem. (B.2), $t_{ij} \leq 1$. Thus only $t = 0$ and $t = 1$ occur, and the corresponding pairwise joint wipe-out probabilities are

$$q_1(k) := \Pr(X_i^{(k)} X_j^{(k)} = 1 \mid t_{ij} = 1) \quad (19)$$

$$= \Pr(|H_i \cup H_j| = 2r - 1 \text{ all fail}) \quad (20)$$

$$= \frac{(k)_{2r-1}}{(N)_{2r-1}}, \quad (21)$$

$$q_0(k) := \Pr(X_i^{(k)} X_j^{(k)} = 1 \mid t_{ij} = 0) = \frac{(k)_{2r}}{(N)_{2r}}, \quad (22)$$

where q_1 is joint wipe-out probability between overlapping types and q_0 is that of between non-overlapping types.

Let C_t be the number of ordered pairs (i, j) with $|H_i \cap H_j| = t$. Because each group hosts exactly r types (Def. (B.1)) and Lem. (B.2) precludes double-counting across multiple shared hosts,

$$C_1 = Nr(r-1), \quad C_t = 0 \text{ for all } t \geq 2. \quad (23)$$

Moreover, for any $J \subset [N]$ with $|J| = r$, the number i_J of host sets intersecting J satisfies

$$i_J \leq r \cdot |J| = r^2, \quad (24)$$

since each group in J hosts r types.

Now, to verify if wipe-out statistics satisfy Barbour & Eagleson (1983, Thm. 1), let

$$k = k_N(x) := \lfloor x N^{1-1/r} \rfloor, \quad x \geq 0 \text{ fixed.} \quad (25)$$

Then, as $k/N = x N^{-1/r} (1 + o(1))$ and, from Eq. (17),

$$p_k = \left(\frac{k}{N} \right)^r (1 + o(1)) = \frac{x^r}{N} (1 + o(1)), \quad \lambda_k = Np_k \rightarrow x^r. \quad (26)$$

We now check the six sufficient conditions of (Barbour & Eagleson, 1983, Thm. 1):

$$(i) \quad Np_k \rightarrow x^r. \quad (26)$$

$$(ii) \quad (N^{-1} \max i_J \leq r^2/N \rightarrow 0. \quad (24)$$

$$(iii) \quad N^{1-r} \rightarrow 0 \text{ for fixed } r \geq 2.$$

$$(iv) \quad C_1 q_1 \rightarrow 0. \quad (23) \quad (21),$$

$$\begin{aligned} C_1 q_1(k) &= Nr(r-1) \left(\frac{k}{N} \right)^{2r-1} (1 + o(1)) \\ &= r(r-1) x^{2r-1} N^{1-(2r-1)/r} (1 + o(1)) \\ &= \mathcal{O}(N^{1/r-1}) \rightarrow 0. \end{aligned}$$

$$(v) \quad N^2 q_1 N^{-1} \rightarrow 0. \quad (21)$$

$$\begin{aligned} N q_1(k) &= N q_1(k) = N \left(\frac{k}{N} \right)^{2r-1} (1 + o(1)) \\ &= x^{2r-1} N^{1-(2r-1)/r} (1 + o(1)) = \mathcal{O}(N^{1/r-1}) \rightarrow 0. \end{aligned}$$

$$(vi) \quad N|q_0 - p_k^2| \rightarrow 0. \quad (22) \quad (17) \quad (26)$$

$$\begin{aligned} \frac{q_0(k)}{p_k^2} &= \frac{(k)_{2r} (N)_r^2}{(N)_{2r} (k)_r^2} = \frac{(k-r)_r}{(k)_r} \cdot \frac{(N)_r}{(N-r)_r} \\ &= 1 + \mathcal{O}\left(\frac{1}{k} + \frac{1}{N}\right), \end{aligned}$$

and for $k \rightarrow \infty$,

$$N|q_0 - p_k^2| = N \cdot \mathcal{O}(N^{-2}) \cdot \mathcal{O}\left(\frac{1}{k} + \frac{1}{N}\right) \rightarrow 0.$$

All conditions of (Barbour & Eagleson, 1983, Thm. 1) are satisfied, therefore $W_{k_N(x)} \Rightarrow \text{Poisson}(x^r)$, and

$$P_{k_N(x)} = \Pr\{W_{k_N(x)} = 0\} \rightarrow e^{-x^r}. \quad (27)$$

Using Eq.-(27) and a Riemann-sum argument with mesh $N^{-(1-1/r)}$,

$$\frac{\mu(N, r)}{N^{1-1/r}} = \sum_{k=0}^{N-1} \frac{1}{N^{1-1/r}} P_k \rightarrow \int_0^\infty e^{-x^r} dx. \quad (28)$$

Finally, with the substitution $u = x^r$ (so $dx = \frac{1}{r} u^{1/r-1} du$),

$$\int_0^\infty e^{-x^r} dx = \frac{1}{r} \int_0^\infty e^{-u} u^{1/r-1} du = \frac{1}{r} \Gamma\left(\frac{1}{r}\right). \quad (29)$$

Therefore,

$$\mu(N, r) = \frac{\Gamma(1/r)}{r} N^{1-1/r} (1 + o(1)), \quad (30)$$

which proves Thm. (4.1). $\square$

B.3. Proof of Theorem 4.2

Proof. Let $q_k \in \{0, 1\}$ denote the indicator that patch compute is needed on the $(k+1)^{\text{th}}$ failure that takes the count from k to $k+1$. We approximate the average overhead before wipe-out by averaging over the per-step stacks as:

$$S(N, r) \approx \frac{1}{\mu} \sum_{k=0}^{\lfloor \mu \rfloor - 1} \left(\mathbb{E}[S(U_k)] + \mathbb{E}[q_k] \right), \quad (31)$$

where $\mu = \mathbb{E}[F]$ is the average failure count to the first wipe-out of Thm. (4.1).

First, we show $\mathbb{E}[S(U_k)] \approx c(k)$ with Hall's Marriage Theorem (Hall, 1987). What we want to show is that for failure count k that $k \leq \mu(N, r)$, the strict inequality $S(U_k) > c(k)$ occurs with negligible probability.

Fix $s := c(k)$ and form the bipartite incidence graph between types and surviving groups: type i is adjacent to group w iff group w is surviving and also hosting type i ($w \in H_i \cap U_k$). Allowing *free* reordering within each group means that a group can be assigned up to s types over the first s stacks. Equivalently, replicate each group $w \in U_k$ into s distinct copies $(w, 1), \dots, (w, s)$ and set

$$L_i := (H_i \cap U_k) \times [s] \subseteq U_k \times [s],$$

the set of all feasible slots to which type i may be assigned (one of its surviving hosts, at one of the first s stacks). Then $S(U_k) \leq s$ is equivalent to the existence of a complete system of distinct representatives (C.D.R.) for $\{L_i\}_{i \in [N]}$; mathematically explained, an injection that can be written as $\phi : [N] \rightarrow U_k \times [s]$ such that $\phi(i) \in L_i$ for all i .

By Hall's Marriage Theorem (Hall, 1987), such an injection exists if and only if for every $E \subseteq [N]$,

$$|E| \leq \left| \bigcup_{i \in E} L_i \right| = s \cdot \left| \bigcup_{i \in E} (H_i \cap U_k) \right|. \quad (32)$$

Therefore $S(U_k) > s$ implies that (32) fails for some witness E . Let the set of groups hosting the subset E of shard types as R :

$$R := \bigcup_{i \in A} (H_i \cap U_k) \subseteq U_k, \quad b := |R|.$$

Then $|E| > s|R|$ and every $i \in E$ satisfies $H_i \cap U_k \subseteq R$. Define the number of shards that *must* be computed by groups in R as $C_R(k)$:

$$C_R(k) := |\{i \in [N] : H_i \cap U_k \subseteq R\}|.$$

We have the implication

$$\{S(U_k) > s\} \subseteq \bigcup_{R \subseteq U_k} \{C_R(k) > s|R|\}, \quad (33)$$

which means that when R is *overloaded* with too many number of shards ($C_R(k)$) it is mandated to compute, then Eq. (32) is to be violated hence $S(U_k) > c(k)$.

Now we bound the RHS in Eq. (33) to formulate the cases where Eq. (32) is violated. Write $x := k/N$ and recall that $|H_i| = r$ and, by Lem. (B.2), $t_{\max} \leq 1$.

Case 1. Single-group pinned overload ($b = 1$). First case $S(U_k) > c(k)$ comes true (Eq. (32) violated), is when a group is mandated to compute certain shards; let us call it as the single-group pinned overload $O_w(k)$: For $w \in U_k$ define the pinned load

$$O_w(k) := |\{i : H_i \cap U_k = \{w\}\}|.$$

If $O_w(k) \geq s+1$ ($s := c(k)$), then $R = \{w\}$ violates Eq. (33) hence Eq. (32). Conditional on $w \in U_k$, for any type $i \in T_w$ (group w 's type set), i is pinned to w exactly when the other $r-1$ hosts of i all fail, hence

$$\begin{aligned} q_{k,N} &:= \Pr(H_i \cap U_k = \{w\} \mid w \in U_k) \\ &= \frac{(k)_{r-1}}{(N-1)_{r-1}} = x^{r-1}(1+o(1)) \quad (k < N). \end{aligned} \quad (34)$$

From Lem. (B.2) ($t_{\max} \leq 1$), a binomial tail bound for $O_w(k)$ is valid and a union bound over at most N groups gives

$$\Pr(\exists w \in U_k : O_w(k) \geq s+1) \leq N \binom{r}{s+1} q_{k,N}^{s+1}. \quad (35)$$

On the first-wipe-out scale $k \asymp \mu = \mathcal{O}(N^{1-1/r})$, we have $x \asymp N^{-1/r}$ and hence $q_{k,N} \asymp N^{-(r-1)/r}$, so the RHS of (35) behaves like $N^{1-(s+1)(r-1)/r}$ up to constants, which vanishes for all fixed $r \geq 2$ and $s = c(k) \geq 2$. Thus *Case 1* is negligible.

Case 2. Multi-group Hall witnesses ($b \geq 2$). Overload may not only happen to a single group, but on multiple groups as well. Fix $R \subseteq U_k$ with $|R| = b \geq 2$. A type counted in $C_R(k)$ must have all its surviving hosts inside R . Such a type either (i) has exactly one surviving host in R (it is pinned to some $w \in R$), or (ii) has at least two surviving hosts in R . Hence,

$$\mathbb{E}[C_R(k)] \leq rb \cdot q_{k,N} + \binom{b}{2} \cdot \frac{(k)_{r-2}}{(N-2)_{r-2}}. \quad (36)$$

The first term counts pinned types across the rb types hosted by R . For the second term, note that if two groups shared two distinct types, it contradicts $t_{\max} \leq 1$; hence each group-pair in R can jointly host at most one type, and for such a type to have all other $r-2$ hosts failed has probability at most $(k)_{r-2}/(N-2)_{r-2}$.

For $k \leq \mu$ we have $x = \mathcal{O}(N^{-1/r})$, so both $q_{k,N}$ and $(k)_{r-2}/(N-2)_{r-2}$ vanish polynomially in N , and in particular $\mathbb{E}[C_R(k)] = o(b)$ uniformly over $b \leq m$. Since

$C_R(k)$ is a sum of indicators under sampling without replacement, a Chernoff bound (Chernoff, 1952; Hoeffding, 1963) applies and yields

$$\Pr(C_R(k) \geq sb) \leq \left(\frac{e \mathbb{E}[C_R(k)]}{sb} \right)^{sb} = N^{-\Omega(b)}. \quad (37)$$

Union bounding Eq. (37) over all $R \subseteq U_k$ and all b implies

$$\Pr(S(U_k) > s) = o(1) \quad \text{for all } k = 0, 1, \dots, \lfloor \mu \rfloor - 1.$$

Together with the deterministic lower bound $S(U_k) \geq s = c(k)$, we obtain

$$\mathbb{E}[S(U_k)] = c(k) + o(1) \quad (k \leq \mu). \quad (38)$$

Therefore, we show $\mathbb{E}[S(U_k)] \approx c(k)$. Now we compute the expectation value of the number of patch computes.

Fix k and set $s := c(k)$, and set n_k as

$$n_k = c(k)(N - k), \quad (39)$$

the number of stacks of computation before the all-reduce attempt. Let d_i be the number of how many type i appears among the n_k slots; then $d_i \geq 1$ and $\sum_{i=0}^{N-1} d_i = n_k$. Let $u_k := |\{i : d_i = 1\}|$ be the number of singleton types: types that appear exactly once among the n_k slots. Since every non-singleton types appear in n_k slots at least 2,

$$\begin{aligned} n_k &= \sum_i d_i \geq u_k + 2(N - u_k) = 2N - u_k \\ \implies u_k &\geq \max\{0, 2N - n_k\}. \end{aligned}$$

As each singleton type corresponds to exactly one slot, when the newly failed group is removed, the s slots computed by that group in the current step are lost, and patch-compute is needed iff at least one of those lost slots was singleton.

Define the singleton-slot fraction as

$$\rho_k := \frac{\max\{0, 2N - n_k\}}{n_k}. \quad (40)$$

Under SPARE's minimal movement reordering by MCMF algorithm (Goldberg & Tarjan, 1990), the singleton slots are typically packed, correlated across groups rather than scattered independently across the s slots of each group. Hence the probability that a uniformly random newly failed group hits at least one singleton is well-approximated to first order by the singleton-slot fraction itself:

$$\Pr(\text{patch compute at failure } k) \approx \rho_k. \quad (41)$$

Therefore, substituting (38) and (41) into (31) yields

$$\bar{S}(N, r) \approx \frac{1}{\lfloor \mu \rfloor} \sum_{k=0}^{\lfloor \mu \rfloor - 1} (c(k) + \rho_k),$$

with $n_k = c(k)(N - k)$ and $\rho_k = \max\{0, 2N - n_k\}/n_k$, proving Thm. (4.2). $\square$

B.4. Proof of Theorem 4.3

Proof. Recall that the normalized time-to-train of SPARE+CKPT is Eq. (7)

$$J(r) := \frac{\text{time-to-train}}{T_0} = \frac{\bar{S}(N, r)}{A^*(\mu(N, r)m)} \quad (42)$$

where m is the MTBF between node failures, $\mu(N, r)$ is the average failure count to first wipe-out from Thm. (4.1), and $A^*(\cdot)$ is the maximal availability obtained by the optimal CKPT period of Eq. (1),(2).

Note that the patch term in $\bar{S}(N, r)$ (5) is always bounded:

$$0 \leq 1 - (1 - \rho_k)^{c(k)} \leq 1, \quad (43)$$

from $0 \leq \rho_k \leq 1$. Therefore, the capacity term $\mathbb{E}[S(U_k)] \approx c(k)$ dominates $\bar{S}$, which increases as a step-function:

$$\begin{aligned} c(k) &= 2 \quad \text{for } 1 \leq k \leq \frac{N}{2}, \\ c(k) &= 3 \quad \text{for } \frac{N}{2} < k \leq \frac{2N}{3}, \\ c(k) &= 4 \quad \text{for } \frac{2N}{3} < k \leq \frac{3N}{4}, \end{aligned}$$

and so on. Thus, as long as the wipe-out threshold $\mu(N, r)$ is below $N/2$, the $\mathbb{E}[S(U_k)]$ contribution in (5) stays pinned at ≈ 2 , and the only variation in $\bar{S}(N, r)$ comes from the bounded patch term (43). Once $\mu(N, r)$ exceeds $N/2$, a nontrivial fraction of indices k in the sum (5) satisfy $k > N/2$, forcing $c(k) \geq 3$ and thereby increasing $\bar{S}(N, r)$ by an $\Omega(1)$ amount while the patch term is bounded ≤ 1 : the increase rate of the numerator of $J(r)$ (7): $\bar{S}(N, r)$ (5), jumps up after $k > \frac{N}{2}$.

Now we look at the denominator of $J(r)$ (7): $A^*(\mu m)$ (2). Because $A^*(T_f)$ increases monotonic in T_f , and $T_f = \mu(N, r)m$, increasing r ($\mu(N, r)$ increases monotonic by r) improves the denominator in (7).

On the other hand, (5) shows that $\bar{S}(N, r)$ remains of constant order $\approx 2 + o(1)$ until the first capacity bump at $\mu(N, r) \approx N/2$, after which $\bar{S}(N, r)$ must increase because $c(k)$ becomes 3 for an increasing fraction of k . Therefore, the minimizer of $J(r)$ occurs at the *smallest* redundancy r that pushes $\mu(N, r)$ to this first capacity transition,

$$\mu(N, r^*) \approx \frac{N}{2}, \quad (44)$$

since below r^* one can still increase availability without paying the $c(k) \geq 3$ penalty on $\bar{S}(N, r)$.

Now we solve $\mu(N, r^*) \approx \frac{N}{2}$ to find optimal redundancy r^* that minimizes $J(r)$. Let $\varepsilon := 1/r$ and rewrite $\mu(N, r)$ (3) as

$$\frac{\mu(N, r)}{N} = \Gamma(\varepsilon) \varepsilon N^{-\varepsilon}.$$

Imposing $\mu(N, r^*) \approx \frac{N}{2}$ gives

$$\Gamma(\varepsilon) \varepsilon N^{-\varepsilon} \approx \frac{1}{2}. \quad (45)$$

For $\varepsilon \ll 1$, we use the standard expansion

$$\Gamma(\varepsilon) = \frac{1}{\varepsilon} - \gamma + \mathcal{O}(\varepsilon),$$

where $\gamma \approx 0.57721$ is the Euler-Mascheroni constant (NIST, 2025). Multiplying by ε yields

$$\Gamma(\varepsilon) \varepsilon = 1 - \gamma \varepsilon + \mathcal{O}(\varepsilon^2).$$

Substituting into (45) and taking logarithms,

$$\log(1 - \gamma \varepsilon + \mathcal{O}(\varepsilon^2)) - \varepsilon \log N \approx -\log 2.$$

Using $\log(1 - x) = -x + \mathcal{O}(x^2)$ gives

$$-(\gamma + \log N) \varepsilon + \mathcal{O}(\varepsilon^2) \approx -\log 2,$$

hence

$$\begin{aligned} \varepsilon &\approx \frac{\log 2}{\log N + \gamma} \implies r^* = \frac{1}{\varepsilon} \approx \frac{\log N + \gamma}{\log 2} \\ &= \log_2 N + \frac{\gamma}{\ln 2}. \end{aligned}$$

Finally, since r is an integer, we take the nearest-integer (or floor) version:

$$r^* \approx \left\lfloor \log_2 N + \frac{\gamma}{\ln 2} \right\rfloor \approx \left\lfloor \log_2 N + 0.833 \right\rfloor,$$

proving Thm. (4.3). $\square$

C. Monte-Carlo Simulation Results

Table 4. Simulation results for $N = 200$ over 1000 trials.

r	$\mu(N, r)$	$\mu(N, r)$	$\mathbb{E}[S(U_k)]$	$\mathbb{E}[S(U_k)]$
$r = 2$	12.5	13.2	1.84	1.90
$r = 3$	30.5	31.3	1.93	1.96
$r = 4$	48.2	49.8	1.97	1.98
$r = 5$	63.6	65.3	1.96	1.98
$r = 6$	76.7	78.5	1.97	1.99
$r = 7$	87.8	89.7	1.97	2.00
$r = 8$	97.1	99.3	1.99	2.03
$r = 9$	105.1	106.9	2.03	2.07
$r = 10$	112.0	113.6	2.09	2.11
$r = 11$	118.0	120.9	2.14	2.16
$r = 12$	123.2	126.3	2.17	2.20

Table 5. Simulation results for $N = 600$ over 1000 trials.

r	$\mu(N, r)$	$\mu(N, r)$	$\mathbb{E}[S(U_k)]$	$\mathbb{E}[S(U_k)]$
$r = 2$	21.7	22.5	1.89	1.94
$r = 3$	63.5	65.3	1.97	1.98
$r = 4$	109.9	108.9	1.97	1.99
$r = 5$	153.3	154.6	1.99	1.99
$r = 6$	191.7	194.8	1.99	2.00
$r = 7$	225.1	227.2	2.00	2.00
$r = 8$	254.0	254.9	1.99	2.00
$r = 9$	279.1	281.4	2.00	2.02
$r = 10$	301.1	302.3	2.00	2.04
$r = 11$	320.4	324.8	2.05	2.08
$r = 12$	337.4	340.0	2.10	2.11
$r = 13$	352.5	355.3	2.14	2.15
$r = 14$	366.1	366.8	2.17	2.18
$r = 15$	378.2	382.1	2.20	2.21
$r = 16$	389.2	393.4	2.22	2.25
$r = 17$	399.2	400.6	2.24	2.27
$r = 18$	408.3	412.6	2.28	2.31
$r = 19$	416.6	420.2	2.31	2.33
$r = 20$	424.2	426.4	2.34	2.36

Table 6. Simulation results for $N = 1000$ over 1000 trials.

r	$\mu(N, r)$	$\mu(N, r)$	$\mathbb{E}[S(U_k)]$	$\mathbb{E}[S(U_k)]$
$r = 2$	28.0	28.6	1.96	1.95
$r = 3$	89.3	89.7	1.98	1.99
$r = 4$	161.2	163.2	1.99	1.99
$r = 5$	230.6	230.4	1.99	2.00
$r = 6$	293.4	296.3	1.99	2.00
$r = 7$	348.7	349.8	1.99	2.00
$r = 8$	397.1	399.3	2.00	2.00
$r = 9$	439.5	443.6	2.00	2.00
$r = 10$	476.8	477.2	1.99	2.02
$r = 11$	509.7	510.2	2.01	2.05
$r = 12$	538.9	543.0	2.06	2.08
$r = 13$	564.9	568.1	2.11	2.12
$r = 14$	588.3	592.3	2.15	2.15
$r = 15$	609.3	608.1	2.17	2.17
$r = 16$	628.3	633.1	2.20	2.21
$r = 17$	645.6	647.3	2.22	2.23
$r = 18$	661.4	663.7	2.24	2.26
$r = 19$	675.9	682.4	2.27	2.30
$r = 20$	689.2	691.6	2.30	2.32
$r = 21$	701.5	704.9	2.33	2.35
$r = 22$	712.8	714.4	2.36	2.37
$r = 23$	723.3	724.6	2.38	2.39
$r = 24$	733.1	736.2	2.40	2.42
$r = 25$	742.2	745.8	2.42	2.44
$r = 26$	750.7	751.9	2.44	2.46

Table 4 ($N = 200$), Table 5 ($N = 600$), Table 6 ($N = 1000$) show the Monte-Carlo simulation results on the average failure count $\mu(N, r)$ (3), and the expectation value of all-reduce stack each scheme with 1,000 trials. Columns of red color are the theoretical values from the formula Eq. (3) and Eq. (6), and the black colored are the simulation results. The results are highly consistent with Thm. (4.1) and Thm. (4.2): Across $N \in \{200, 600, 1000\}$ and all tested redundancies, Monte Carlo results match the closed-form formulas with Mean Absolute Percentage Error 1.13% for $\mu(N, r)$, and 0.60% for average all-reduce stack, with correlations ≥ 0.996 (worst-case relative error $\leq 5.06\%$). See the file `reordering.ipynb` in Supplementary Materials for the code and the detailed simulation results. Simulation is implemented by emulating the trails of random independent failures and corresponding reordering events before the first wipe-out.

D. Reordering Controller Algorithm

We detail how HK-FIXED and HK-FREE (Hopcroft & Karp, 1973), and MCMF (Goldberg & Tarjan, 1990) implement Alg. (2), reordering controller, and why their costs are negligible at $N \sim \mathcal{O}(10^{2-3})$.

Bipartite feasibility model. Define a bipartite graph with left vertices $\mathcal{L} = [N]$ (types) and right vertices $\mathcal{R} = U_k \times [S_A]$ (slots of computation among survivors). An edge $(i, (w, t))$ means survivor w can compute type i at stack t . A size- N matching is equivalent to collecting all N types by depth S_A (one type per slot, every type covered).

HK-FIXED (Phase 0: validate committed stacks). Given the committed depth S_A and the current per-group order $\text{stk}[w]$, construct edges $(\text{stk}[w][t], (w, t))$ for all $w \in U_k, t \leq S_A$. Run Hopcroft–Karp (Hopcroft & Karp, 1973) to test if the maximum matching has size N . If yes, all-reduce at S_A succeeds without reordering; otherwise proceed to Phase 1.

HK-FREE (Phase 1: find minimal $S(U_k)$ under free reordering). For candidate $S = S_A, S_A + 1, \dots, r$, allow free permutation within each group stack: add edges $(i, (w, t))$ iff $w \in H_i \cap U_k$. Run Hopcroft–Karp (Hopcroft & Karp, 1973); the first S with matching size N is $S(U_k)$. If no $S \leq r$ works, some type(s) has no surviving host (wipe-out) and the controller restarts.

MCMF (Phase 2: minimum-movement reordering at $S(U_k)$). At $S^* := S(U_k)$, we seek a feasible assignment that changes stacks minimally. Use the same bipartite graph but assign each edge a movement cost as 0 if already at position t , 1 if making a movement. Compute a min-cost size- N assignment via a standard min cost max flow al-

gorithm (Goldberg & Tarjan, 1990), then update each survivors’ stack accordingly.

Complexity and latency. Hopcroft–Karp (Hopcroft & Karp, 1973) runs in $\mathcal{O}(E\sqrt{V})$ time where E is the number of edges and V is the number of vertexes (nodes). Here $V = N + (N - k)S$ and $E \leq N \cdot r \cdot S$, since each type has $\leq r$ surviving hosts and S slots to be assigned per each group. For $N \sim \mathcal{O}(10^{2-3})$ and $r \leq \mathcal{O}(10^{0-1})$, and $S \approx 2-3$, these graphs are sparse and small, so HK-FIXED/HK-FREE are sub-100ms in compiled implementations. MCMF is run only upon failures and restricted to types and slots impacted by the failure, making its practical cost likewise negligible; we conservatively model the controller as 0.1s overhead per failure in our simulations.

Potential Acceleration by Binary Search The HK-FREE feasibility predicate is monotone in the candidate depth S : if all types are collectible by depth S , then they remain collectible for any larger depth $S' > S$. Therefore, the linear scan over S in Alg. 2 can be replaced by a standard binary search over the ordered depth range, reducing the number of HK-FREE calls from $\mathcal{O}(r)$ to $\mathcal{O}(\log r)$ (Cormen et al., 2022). In our operating regime, however, the practical speedup is limited because the minimal feasible depth is typically small, around 2–3, and each HK-FREE call is already inexpensive at $N \sim 10^2-10^3$.

E. Why Discrete-Event Simulation?

E.1. Simulator Choice

Our simulator evaluates long-horizon fault-recovery dynamics of a large-scale training system, rather than only steady-state throughput under a fixed training configuration. The events to emulate include compute, gradient synchronization, failed all-reduce, failure detection, communicator shrink, reordering, checkpointing, and global restart. We therefore use a discrete-event simulator developed upon FedDES (Chen et al., 2025), a large parallel system simulation toolkit built on top of SimGrid (Casanova et al., 2013) which matches our system-level abstraction and fault-recovery evaluation target.

ASTRA-sim (Rashidi et al., 2020; Won et al., 2023) and Calculon (Isaev et al., 2023) are strong alternatives for ML-system performance modeling. ASTRA-sim is well suited for distributed-training SW/HW co-design and detailed performance exploration under specified model, parallelism, memory, and network configurations. Calculon provides a fast analytical model for LLM training performance and algorithm-system co-design. These tools mainly target inner-loop questions such as throughput, communication bottlenecks, and parallelism strategy. In contrast, SPARe requires an outer-loop reliability evaluation: how failures

accumulate and global restarts are bypassed, and how much wall-clock time is saved over a stochastic failure trajectory. FedDES/SimGrid is therefore a better fit for our evaluation target.

E.2. SimGrid Validation and Use in HPC Studies

Prior work has validated SimGrid’s modeling accuracy and scalability in distributed and HPC-relevant settings. Examples include flow-level TCP network models (Velho et al., 2013), scalable simulation of distributed applications and platforms (Casanova et al., 2014), MPI application simulation through SMPI (Degomme et al., 2017), and SimGrid-based data-transport simulation for in situ workflows (Suter, 2025). These works support SimGrid as an appropriate abstraction for system-level timing studies involving computation, communication, and resource contention.

SimGrid-based simulation has also been used as a main evaluation vehicle in peer-reviewed HPC venues. Examples include simulation-driven scheduling policies at SC (Carastan-Santos & De Camargo, 2017), volunteer-computing simulation at HPDC (Donassolo et al., 2010), bandwidth-aware scheduling at IPDPS (Beaumont & Rejeb, 2010), large-scale backfilling studies at CCGrid (Carastan-Santos et al., 2019), and edge devices system studies for federated learning at SEC (Chen et al., 2025). This precedent supports our SimGrid-based discrete-event evaluation of SPARe, especially because direct experimentation at 100k–600k GPU scale is beyond realistic academic access.

F. Additional Figures

F.1. Communicator Topology

See Fig. 9.

F.2. Simulation Flowchart

See Fig. 10.

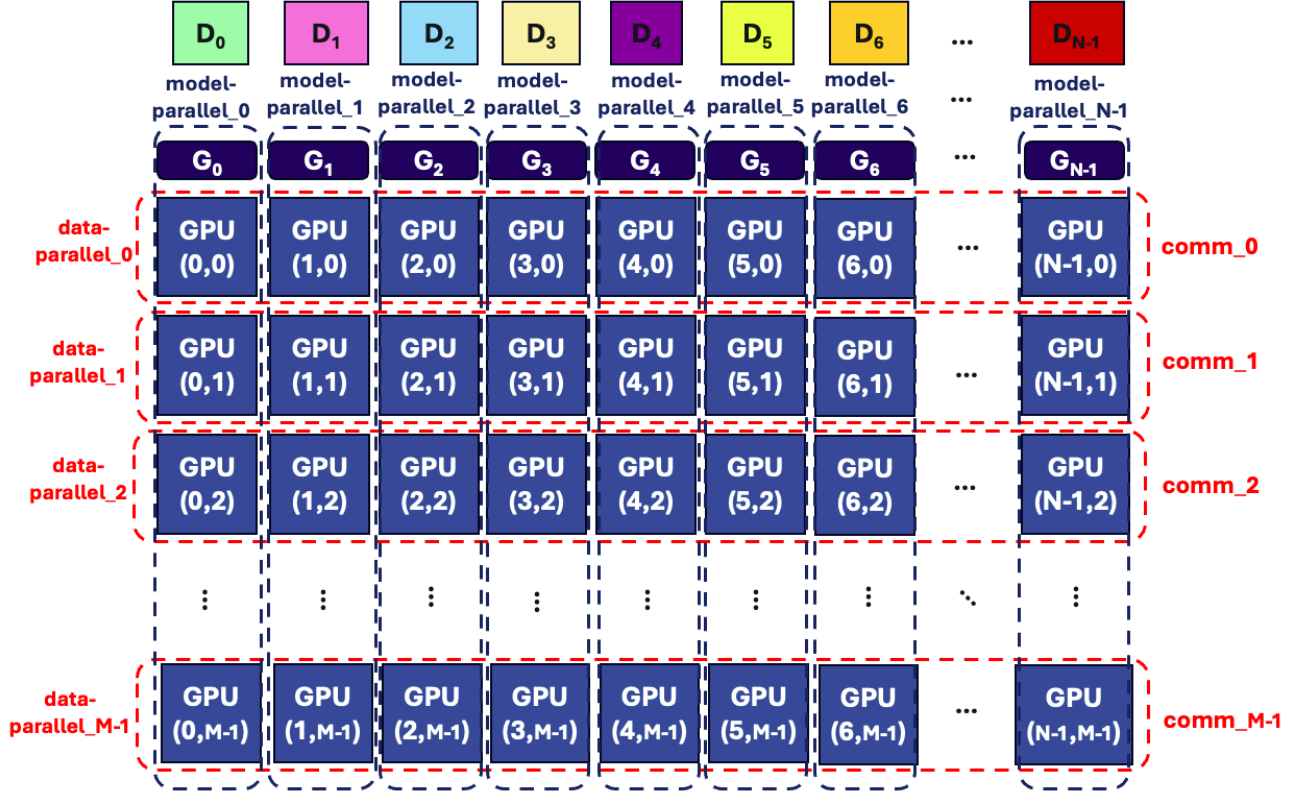


Figure 9. Communicator topology on data/model-parallel groups based on the descriptions of Sec. 2.1.

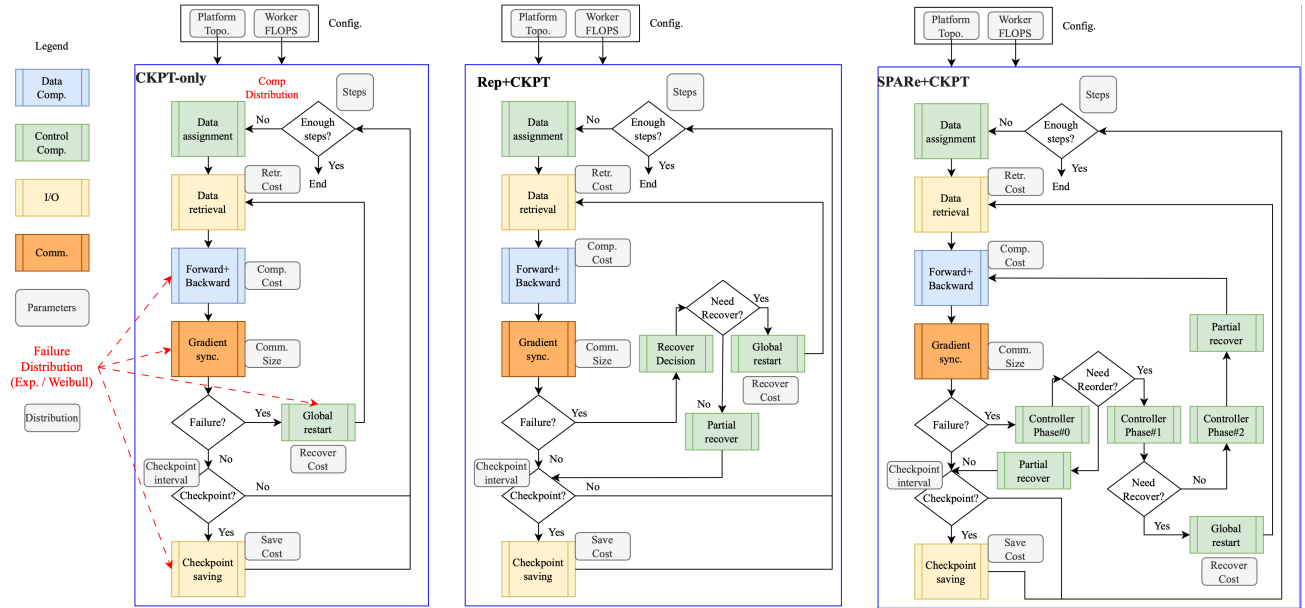


Figure 10. Simulation flowchart of CKPT-only, Rep+CKPT, and SPARE+CKPT